Fiche de lecture : dilemme du prisonnier répété, tournoi d'Axelrod et agents LLM

Projet intégrateur n° 6, module Théorie des Jeux et Systèmes Multi-Agents (Pr A. Jraifi), ENSA Safi, 2026-2027

Groupe 6 : MARZAQ Jihane, SADOUR Assia, TSETSE Komla Edem Wilfried, ZERBAOUI Marwane

Phase 0, semaine 1, séance du 11 novembre 2026

1. Le modèle et ses formules

Le jeu de base oppose deux joueurs qui choisissent chacun de coopérer (C) ou de trahir (D). Les gains par manche sont $(C,C) = (R,R) = (3,3)$, $(C,D) = (S,T) = (0,5)$, $(D,C) = (T,S) = (5,0)$ et $(D,D) = (P,P) = (1,1)$. C'est un dilemme du prisonnier si

$$T > R > P > S \qquad \text{et} \qquad 2R > T + S,$$

ici $5 > 3 > 1 > 0$ et $6 > 5$. La seconde condition empêche que l'alternance (trahir, coopérer) rapporte plus que la coopération mutuelle. Trahir domine strictement coopérer, donc (D,D) est l'unique équilibre de Nash du jeu à une manche, alors que (C,C) le Pareto-domine (3 contre 1 pour chacun).

Horizon fini connu. Par induction à rebours, la dernière manche est un dilemme à une manche, donc on trahit ; la manche précédente n'a alors plus d'enjeu, on trahit aussi, et ainsi de suite jusqu'à la première. Le seul équilibre parfait en sous-jeux est « trahir à chaque manche » : c'est le paradoxe du mille-pattes. Sur les 200 manches du projet, la théorie prédit donc la trahison, et c'est l'écart avec les stratégies conditionnelles que le projet mesure.

Horizon indéterminé. La partie continue après chaque manche avec la probabilité δ. Un joueur qui coopère toujours face à un rancunier (*grim trigger*) obtient $R/(1-\delta)$; s'il trahit une fois, il obtient $T + \delta P/(1-\delta)$. La coopération est stable si

$$\frac{R}{1-\delta} \;\geq\; T + \frac{\delta P}{1-\delta} \;\iff\; \delta \geq \frac{T-R}{T-P} = \frac{1}{2}.$$

Contre tit-for-tat, la déviation « trahir toujours » donne le même seuil, mais la déviation qui alterne $T, S, T, S, \ldots$ rapporte $(T + \delta S)/(1-\delta^2)$ et exige $\delta \geq (T-R)/(R-S) = 2/3$.

Bruit. Chaque coup est inversé avec la probabilité ε, pour chacun des deux joueurs (la « main tremblante » de Selten). Une erreur peut déclencher des représailles en chaîne chez tit-for-tat et le rancunier.

Dynamique du réplicateur. Pour une population de parts x_i et la matrice M des scores moyens par manche (i contre j), le score de la stratégie i est $f_i = (Mx)_i$, la moyenne $\bar{f} = x^\top f$, et

$$x_i(t+1) = x_i(t)\,\frac{f_i}{\bar{f}}.$$

Les parts restent positives et de somme 1, car $\sum_i x_i f_i = \bar{f}$.

2. Hypothèses

L'horizon est de 200 manches par rencontre (annoncé ou non pour les LLM : c'est une variable d'expérience) ; le bruit vaut $\varepsilon \in \{0\,;0{,}01\,;0{,}05\}$; six stratégies : toujours coopérer, toujours trahir, tit-for-tat (coopère puis copie le dernier coup adverse), rancunier, Pavlov (coopère si les deux ont joué pareil au tour précédent), aléatoire. Les joueurs observent les coups exécutés, après bruit. Limites : le modèle est symétrique, à deux joueurs, sans coût de mémoire ; le tournoi dépend de l'ensemble de stratégies choisi, et le réplicateur suppose une population infinie qui se mélange parfaitement.

3. Lien avec le code du cours

Le guide désigne comme socle les fonctions du cours (équilibres de Nash, induction à rebours `backward_induction`, `nash_mixte_2x2`) et l'agent `agent_tj_sma.py` avec ses dix outils. Nous en étendons trois idées : le test d'équilibre de Nash d'un jeu 2×2 sert à vérifier le dilemme à une manche ; l'induction à rebours sert à retrouver, sur n manches, la trahison prédite ; l'agent reçoit **deux nouveaux outils** `@tool`, `tournoi_axelrod(strategies, manches, bruit, seed)` et `replicateur(strategies, generations)`, avec sortie JSON et paramètres plafonnés. Le code du cours n'est pas encore dans notre dépôt : nous partons des conventions du guide et adapterons dès réception.

4. Algorithme en Python

1. Une stratégie est une fonction `(mon_historique, historique_adverse, rng) -> 'C' | 'D'`.
2. `match(s1, s2, manches, bruit, rng)` joue les manches ; à chacune, les deux coups sont tirés, chaque coup est inversé avec la probabilité ε, les gains sont cumulés, et la fonction renvoie les **scores moyens par manche**.
3. `tournoi(strategies, manches=200, bruit=0.0, seed=0)` joue toutes les paires, y compris une stratégie contre elle-même (score d'auto-jeu = moyenne des deux côtés), renvoie la matrice M et le classement par moyenne de ligne.
4. `replicateur(scores, x0, generations)` itère la formule ci-dessus à partir de parts égales.
5. Un joueur LLM suit la même signature : il reçoit l'historique, répond par une ligne `CHOIX : COOPERER` ou `CHOIX : TRAHIR`, toute réponse sans cette ligne est comptée à part. Au moins 10 parties de 10 manches par configuration, avec comparaison en gain moyen par manche.

5. Références

Axelrod, R. (1984), *The Evolution of Cooperation*, Basic Books. Axelrod, R. et Hamilton, W. D. (1981), « The evolution of cooperation », *Science* 211. Nowak, M. et Sigmund, K. (1993), « A strategy of win-stay, lose-shift that outperforms tit-for-tat in the Prisoner's Dilemma game », *Nature* 364. Selten, R. (1975), « Reexamination of the perfectness concept for equilibrium points in extensive games », *Int. J. Game Theory* 4. Fudenberg, D. et Maskin, E. (1986), « The folk theorem in repeated games with discounting or with incomplete information », *Econometrica* 54. Taylor, P. D. et Jonker, L. B. (1978), « Evolutionarily stable strategies and game dynamics », *Math. Biosci.* 40.